POLYTECH DIJON

Introduction à l'IA : optimisation bio-inspirée

TP 2 : Algorithmes gloutons

Objectif du TP

Ce TP a pour objectif de mettre en pratique les principes des algorithmes gloutons à travers l'implémentation de plusieurs problèmes classiques d'optimisation. Il vise notamment à traduire une stratégie gloutonne en programme, à tester son comportement sur différentes instances et à analyser l'influence du critère de sélection sur la qualité des solutions obtenues.

Compétences à valider à l'issue du TP

- ☐ Implémenter un algorithme glouton à partir d'une stratégie de sélection donnée.
- ☐ Identifier et mettre en uvre un critère de sélection adapté à un problème d'optimisation.
- ☐ Tester un algorithme sur différentes instances et analyser les solutions obtenues.
- ☐ Mettre en évidence les limites d'une approche gloutonne et évaluer l'influence du critère de sélection utilisé.

1 Rendu de monnaie



Nous souhaitons développer un algorithme glouton permettant à un distributeur de boissons de rendre la monnaie à un client à partir d'un ensemble de pièces disponibles. L'objectif est de minimiser le nombre total de pièces utilisées.

Le distributeur dispose de pièces de 1, 2, 5, 10, 20 et 50 centimes, ainsi que de pièces de 1 et 2 euros. Afin de simplifier l'implémentation, toutes les valeurs seront exprimées en centimes : les pièces de 1 et 2 euros seront donc représentées respectivement par les valeurs 100 et 200.

1. Implémentez un algorithme glouton qui prend en paramètre une somme à rendre et retourne les pièces à utiliser ainsi que leur quantité.

2. Modifiez l'ensemble des pièces disponibles et construisez un exemple pour lequel l'algorithme glouton précédent ne fournit pas une solution optimale. Comparez la solution obtenue avec une solution optimale.

© Wahabou Abdou — wahabou.abdou@ube.fr — Polytech Dijon 2026 – 2027

2 Réservation d'une salle d'examen

Un Directeur des études souhaite planifier les épreuves pouvant être organisées dans une salle au cours d'une journée. Deux épreuves ne peuvent pas occuper la salle simultanément. L'objectif est de planifier le plus grand nombre possible d'épreuves, et non de maximiser la durée d'utilisation de la salle.

Chaque épreuve est caractérisée par son intitulé, son horaire de début et son horaire de fin. La figure 1 présente un exemple d'épreuves pour lesquelles une réservation de la salle est souhaitée. On peut, par exemple, constater que les épreuves de Maths et d'Informatique sont en conflit et ne peuvent donc pas toutes les deux être planifiées.

L'objectif de cet exercice est d'implémenter un algorithme glouton permettant de planifier un nombre maximal d'épreuves dans la salle.

Une stratégie possible consiste à trier les épreuves par horaires de fin croissants, puis à sélectionner l'épreuve qui se termine le plus tôt. Parmi les épreuves restantes, on sélectionne ensuite celle qui se termine le plus tôt parmi celles qui sont compatibles avec les épreuves déjà retenues. Ce processus est répété tant qu'une nouvelle épreuve peut être ajoutée au planning.

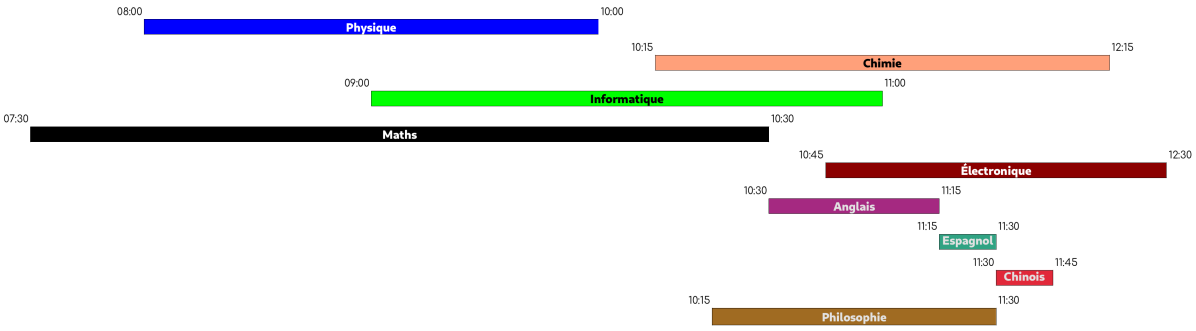


FIGURE 1 – Exemple d'épreuves à planifier

1. Proposez une méthode permettant de déterminer si une épreuve donnée est en conflit avec une autre épreuve. Déduisez-en comment identifier toutes les épreuves d'une liste qui sont en conflit avec une épreuve donnée.

2. Complétez la méthode `eliminerConflits(Epreuve e)` de la classe `ListeEpreuves`. Cette méthode doit supprimer de la liste toutes les épreuves qui sont en conflit avec l'épreuve fournie en paramètre.

3. Testez cette méthode en supprimant toutes les épreuves en conflit avec l'épreuve de Physique. Affichez ensuite la liste des épreuves restantes et vérifiez la cohérence du résultat obtenu.

4. Dans la classe `Main`, implémentez un algorithme glouton permettant de planifier un nombre maximal d'épreuves dans la salle.

5. Modifiez votre algorithme afin d'utiliser un autre critère de sélection que celui fondé sur l'horaire de fin le plus proche. Cette modification pourra nécessiter d'adapter la méthode `compareTo` de la classe `Epreuve`. Testez cette nouvelle stratégie et comparez les résultats obtenus avec ceux de la stratégie précédente.

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026 – 2027

3 Bipartition (exercice bonus)

Soit un ensemble E composé de n éléments numériques. On souhaite répartir ces n éléments entre deux sous-ensembles disjoints E_1 et E_2. L'objectif est d' équilibrer au mieux les sommes de leurs éléments. Idéalement, on souhaite obtenir : :

$$\sum_{x \in E_1} x = \sum_{x \in E_2} x$$

Lorsque cette égalité ne peut pas être atteinte, on cherchera à minimiser lécart entre les deux sommes.

1. Implémentez l'algorithme étudié en TD. Vous pouvez également proposer une autre stratégie, après validation par l'enseignant.

2. Testez successivement vos algorithmes avec les ensembles suivants :

   (a) $E = \{2, 10, 3, 8, 5, 7, 9, 5, 3, 2\}$;

   (b) $E = \{771, 121, 281, 854, 885, 734, 486, 1003, 83, 62\}$.

   Pour chaque instance, comparez les résultats obtenus par les différentes stratégies.

© Wahabou ABDOU — wahabou.abdou@ube.fr — Polytech Dijon 2026 – 2027